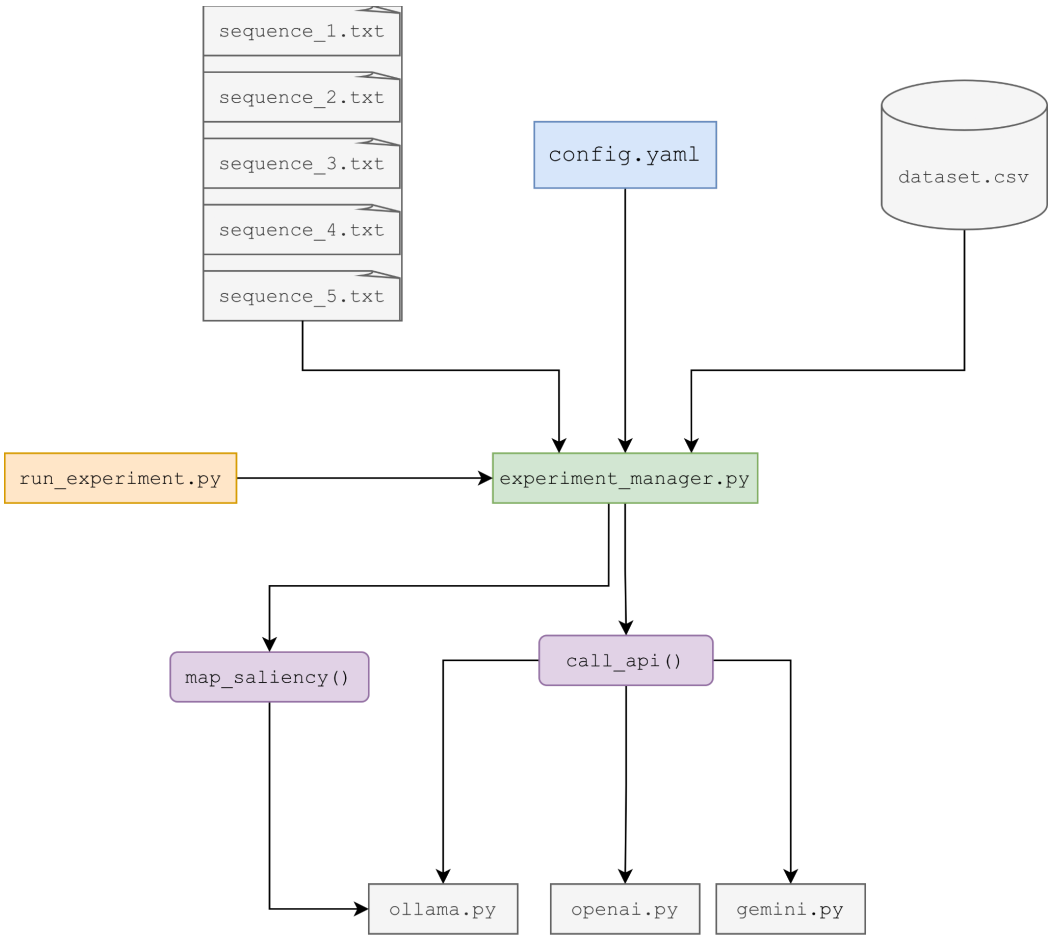
System Design Document: LLM End User

Author: Aarya Bhave bhaveaarya09@gmail.com
Last Updated: May 20, 2026
Status: DRAFT

1. Executive Summary

This document outlines the architecture for an automated, Python-based pipeline designed to simulate LLM subjects for the phishing susceptibility experiment. The system is designed to be highly modular, separating the experimental loop from API routing and data analysis. It batch-processes randomized email sequences , queries multiple local and cloud-based LLM providers, computes text saliency, and aggregates the structured outputs for statistical analysis.

2. System Architecture Diagram



3. Core Components & Responsibilities

3.1. Input & Configuration

- **sequence_1.txt to sequence_5.txt:** The static input files containing the randomly scrambled sequences of the 56 emails. These files define the trial order, cognitive bias, author, and legitimacy status for each simulated run.
- **config.yaml:** The central configuration file. This manages environment variables, API keys, base prompts, and run parameters (such as targeting specific models, enforcing a temperature of 0, and defining the selected sequence).

3.2. Execution & Orchestration

- **run_experiment.py:** The primary entry point of the application. It acts as the high-level bootstrap script, parsing command-line arguments, loading the specified sequence file, and initializing the experimental loop.
- **experiment_manager.py:** The core orchestrator.
    - Ingests settings from config.yaml.
    - Iterates through the loaded sequence of emails.
    - Handles the assembly of the prompt payload (combining instructions, plain text email, and HTML image).
    - Manages the data flow between API calls, processing functions, and storage.

3.3. Model Interaction Layer

- **call_api():** A unified abstraction interface. It takes the standardized payload from the experiment_manager and routes it to the correct backend based on the active configuration.
- **API Wrappers (ollama.py, openai.py, gemini.py):** Dedicated modules that translate the standardized request into the specific schemas required by different LLM providers. This design ensures that swapping between a local open-source model (via Ollama) and a cloud provider (OpenAI/Gemini) requires zero changes to the core experimental logic.

3.4. Processing & Storage

- **map_saliency():** The data extraction and analysis module. It processes the raw text returned by the LLMs to extract the relevant email components. Depending on the token budget, this module executes either the direct extraction of the relevant_strings_dict or manages the leave-one-out word ablation logic to score token relevance.
- **dataset.csv:** The final data sink. A structured, continuously updated file that records the subject_id, trial metadata, selected action (hit/no hit) , and the computed saliency vectors for downstream Chi-Square and Cosine Similarity analyses.

4. Execution Data Flow

1. **Initialization:** run_experiment.py is executed, loading the target sequence (e.g., sequence_1.txt) and invoking experiment_manager.py.
2. **Configuration Load:** experiment_manager.py reads config.yaml to establish connections and format the base prompts.
3. **Trial Execution:** For each email in the sequence:
    - The manager constructs the prompt and invokes call_api().
    - call_api() delegates the request to the configured wrapper (e.g., gemini.py), handling network transmission and rate-limiting.
4. **Response Processing:** The raw LLM response is returned to the manager and immediately passed to map_saliency() to parse out the selected action and the string justifications.
5. **Persistence:** The parsed, structured data row is appended to dataset.csv. The loop continues to the next email in the sequence.